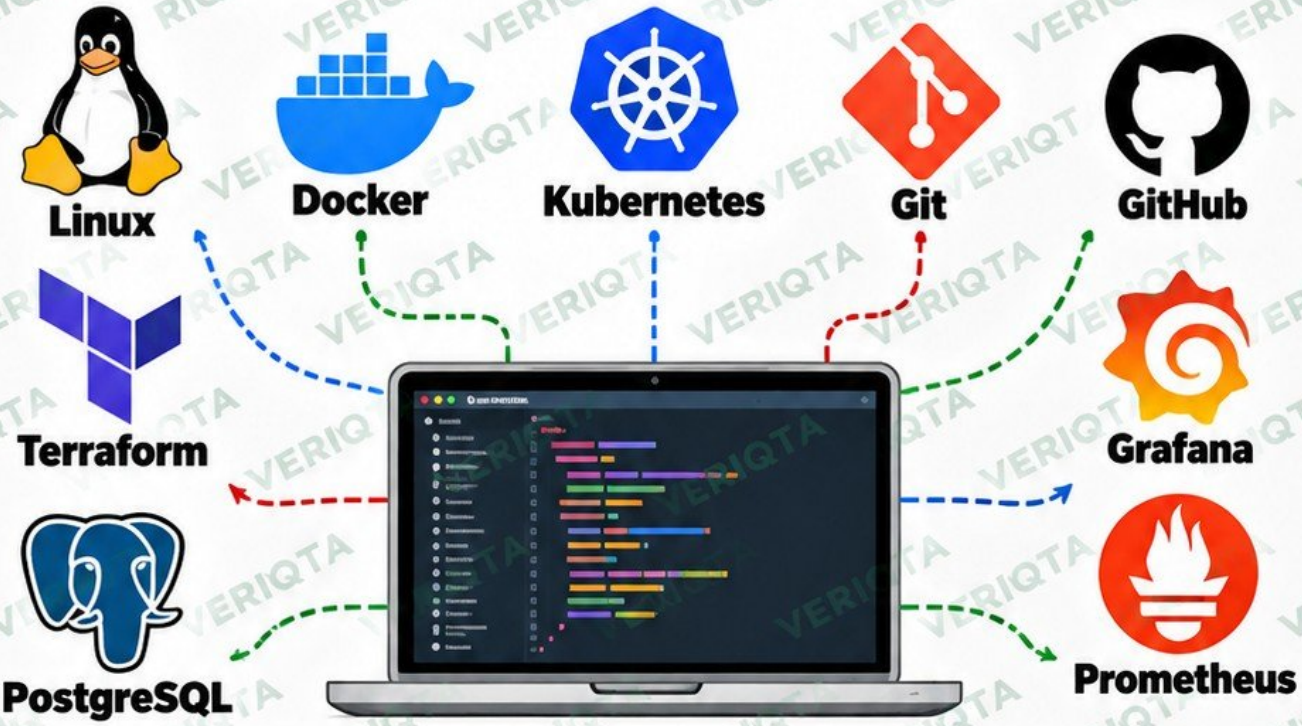
VERIQTA

20 PAGES

How to Set Up Your DevOps Home Lab on a Budget



BUILD
Real Infrastructure with Free Tools

LEARN
By Building Real Projects

GAIN
Practical DevOps Skills for Your Career

@veriqta @veriqta @veriqta @veriqta @veriqta @veriqta

Your DevOps Lab Starts With What You Already Own

1. What Is a DevOps Home Lab?

- A DevOps home lab is a personal learning environment where you install and practice tools used in real engineering work.
- It helps you learn by doing, make mistakes safely, and build practical skills.
- Engineers use home labs to gain experience, test ideas, and prepare for real jobs.

2. What Can You Learn?

- Linux and terminal skills
- Git and GitHub
- Docker and containers
- Kubernetes (local cluster)
- CI/CD with GitHub Actions
- Databases (e.g., PostgreSQL)
- Monitoring (e.g., Prometheus & Grafana)
- Infrastructure as Code (e.g., Terraform)
- Troubleshooting real engineering problems

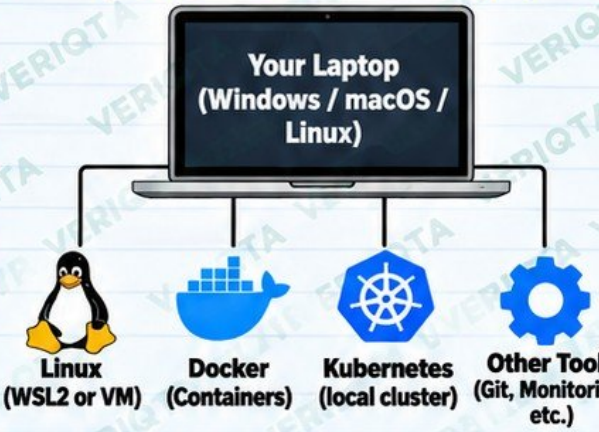
4. Why You Don't Need Expensive Equipment

- You can learn most DevOps tools on a normal laptop.
- No need for physical servers, racks, or data center hardware.
- Most tools are free and lightweight.
- Cloud subscriptions are optional and can be used later when needed.
- Your laptop is enough to build real, practical skills.

6. Practical Exercise

1. Check your CPU: Look at your processor model (e.g., Intel i5, i7, AMD Ryzen).
2. Check your RAM: Windows (Settings > System), macOS (About This Mac), or Linux (free -h).
3. Check your disk space: Windows (This PC), macOS (Storage), or Linux (df -h).
4. Decide which lab setup your computer can support (e.g., containers only, or a VM + containers).

Typical Home Lab Setup



3. Minimum Practical Hardware

Component	Recommended (Minimum)
CPU	4 cores (Intel i5 / AMD Ryzen 5 or better)
RAM	8 GB (16 GB recommended for multiple VMs/containers)
Storage	100 GB free space (SSD prefrered)

Signs Your Computer May Struggle

- Less than 4 GB RAM
- Very slow when running one virtual machine
- Disk space is always almost full
- Your system freezes when running containers
- You cannot run more than one VM comfortably

5. Local Lab vs Cloud Lab

Feature	Local Lab (Laptop)	Cloud Lab (e.g., AWS)
Cost	Free	Can incur charges
Setup time	Fast	Slower (account setup)
Learning focus	Core skills	Real cloud services
Limitations	Limited resources (CPU, RAM)	Usage limits and potential costs
Best use case	Learning and practice	Advanced scenarios and cloud-specific tools

Junior Engineer Tip

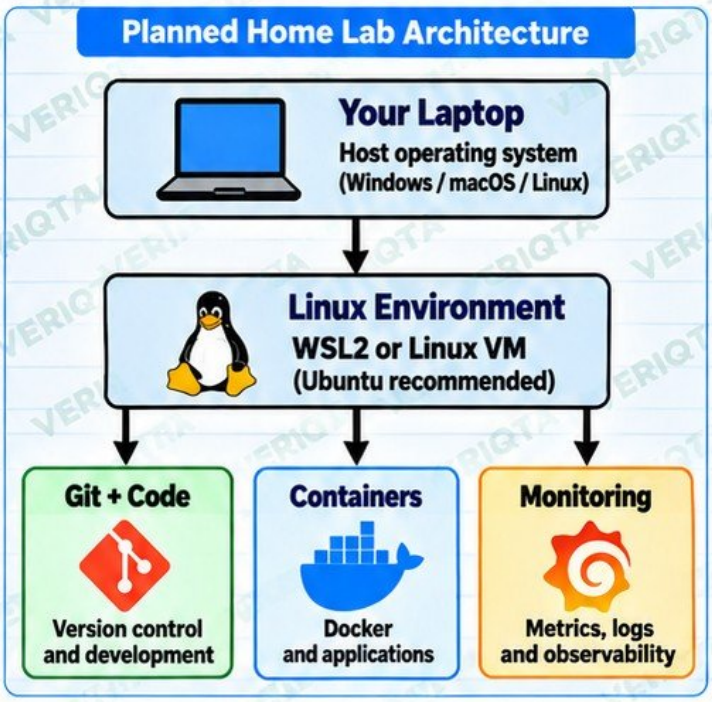
Start small. A reliable lab that you understand is more valuable than expensive infrastructure you cannot troubleshoot.

Design Your Home Lab Before Installing Anything

1. What Components Are Needed?

1. **Host OS** — Your main operating system (Windows, macOS, or Linux).
2. **Linux Environment** — WSL2, Linux VM, or native Linux (where most tools will run).
3. **Containers** — Docker for building and running isolated applications.
4. **Networking** — Local networks, ports, and communication between services.
5. **Development Tools** — Git, code editor, terminal, and other DevOps tools (e.g., Docker, Kubernetes, Terraform).

Planned Home Lab Architecture



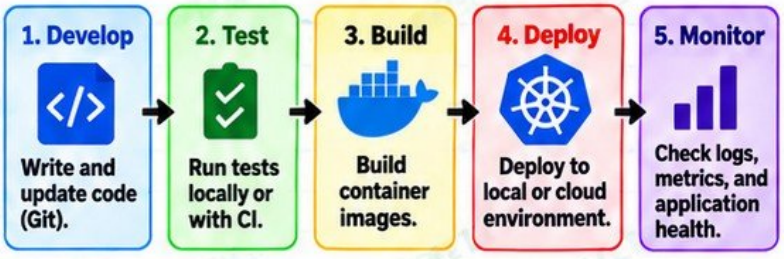
2. What Runs Where?

Runs on Your Laptop (Host OS)	Runs Inside Linux (WSL2 or Linux VM)
Operating system	Linux distribution (e.g., Ubuntu)
Terminal / IDE	All DevOps tools (Docker, Kubernetes, etc.)
Browser	Application code and configuration files
Git (can be on both)	Containers and services
Resource management	Databases, monitoring tools, and test environments

3. Virtual Machine vs Container

Feature	Virtual Machine (VM)	Container
What it is	Full operating system running on your host	Isolated application sharing the host OS
Resource usage	Heavier (more CPU, RAM, and disk)	Lighter (shares host kernel)
Boot time	Slower (minutes)	Faster (seconds)
Isolation	Full OS isolation	Process-level isolation
Typical use	Run another OS (e.g., Ubuntu)	Run applications (e.g., web app, database)
Best for	Learning full OS or testing complex setups	Application development, testing, and deployment

4. Plan Your DevOps Workflow



5. Choose a Lightweight Architecture

- Start with a simple setup that matches your laptop's resources.
- Use containers for most services.
- Add tools gradually as you learn.
- Keep the architecture modular (easy to expand).
- Avoid running too many virtual machines at the same time.
- Use free and open-source tools where possible.

6. Practical Exercise

1. Draw your own home lab architecture.
2. Label each component (host OS, Linux, tools, services).
3. Decide which tools you will install first.
4. Keep it simple and realistic for your current computer.

Junior Engineer Tip

Start with a lightweight design that you understand. A small, reliable lab is more valuable than a complex setup you cannot maintain.

Set Your Budget and Avoid Surprise Costs

1. Why Budgeting Matters

- Cloud resources are powerful but can get expensive if you don't plan.
- A clear budget helps you learn without unexpected charges.
- You can build a fully functional DevOps lab using your existing laptop and free tools.
- Understanding costs early builds good engineering habits.

2. Use What You Already Have



Your Laptop
Use your existing Windows, macOS, or Linux computer.



Free Tools
Use open-source tools and free tiers (e.g., Docker, VS Code, Git, etc.).

3. Separate Essential vs Optional

Essential (Free or Low Cost)	Optional (May Cost Money)
Operating system (Windows/macOS/Linux)	More RAM or SSD storage
Linux environment (WSL2 / VM)	Cloud virtual machines
Docker and containers	Managed databases
Kubernetes (kind or minikube)	Domain name
Git and GitHub (free)	Paid cloud services
Code editor (VS Code)	Additional monitoring tools
Monitoring tools (Prometheus, Grafana)	High-performance hardware

4. Understand Free Tiers, Trials and Quotas

Free Tier
- Limited resources for a certain period.
- Good for learning and testing.
- Example: AWS, Azure, Google Cloud.

Trial
- Temporary access to paid features.
- Usually 30–90 days.
- Can automatically convert to a paid account.

Quotas
- Usage limits on certain services.
- Going over the limit can lead to charges.
- Always check the limits for each service.

5. Why Free Accounts Can Still Charge You

A free account does not mean everything is free. You can still be charged if you:

- Use resources outside the free tier.
- Keep resources running after the trial ends.
- Create large storage or databases.
- Use a public IPv4 address (often charged).
- Transfer large amounts of data.
- Enable paid features by mistake.

6. Common Cost Risks

Resource	Why It Can Cost Money	Example
Virtual Machines	Compute charges based on time and size.	AWS EC2, Azure VM, Google Compute Engine
Public IPv4 Address	Many cloud providers charge for public IPs, even when not used.	AWS Elastic IP, Azure Public IP
Storage	Charges based on amount and type of storage used.	EBS, S3, Azure Disk, Cloud Storage
Data Transfer	Moving data in or out of the cloud can be expensive.	Internet outbound traffic charges
Managed Databases	Charges based on size, usage, and backups.	RDS, Azure Database, Cloud SQL
Additional Services	Features like load balancers, monitoring, logs, or snapshots.	ALB, CloudWatch, Azure Monitor, etc.

7. Local vs Cloud: When to Use Each

Criteria	Local Lab (Laptop)	Cloud Lab
Cost	Free	Can incur charges
Setup time	Fast	Slower (account setup)
Internet required	Not required	Required
Best for	Learning and practice	Advanced scenarios and cloud-specific tools
Limitations	Limited CPU, RAM	Usage limits and billing risk

8. Practical Exercise



1. List the equipment you already have (laptop specs).
2. List the tools you will use (mark free or paid).
3. Create a simple monthly budget (e.g., $0, $5, $10).
4. Research the free tier limits for one cloud provider.
5. Write a cloud spending checklist for your account.

9. Cloud Spending Checklist



- Enable billing alerts.
- Check free tier limits.
- Review running resources regularly.
- Delete unused resources.
- Avoid leaving public IPs, VMs, or databases running.
- Monitor storage and data transfer.



Security Reminder
Billing alerts help you detect spending, but they do not automatically stop every charge. You are responsible for monitoring and removing resources you no longer need.



Junior Engineer Tip
Start small. A reliable lab that you understand is more valuable than expensive infrastructure you cannot troubleshoot.

Install Linux Without Replacing Your Main Operating System

1. What Is Linux?

- Linux is an open-source operating system.
- Ubuntu is a popular Linux distribution used in DevOps and cloud engineering.
- You interact with Linux using a terminal (command line).
- Linux is stable, secure, and widely used in servers, cloud, and containers.

2. WSL2 vs Virtual Machine vs Native Linux

Feature	WSL2 (Windows)	Virtual Machine (VM)	Native Linux (Install on PC)
Runs on	Windows	Windows, macOS, Linux	Replaces your current OS
Performance	Good (lightweight)	Slower (uses more resources)	Best
Ease of setup	Very easy	Moderate	Requires full installation
Use case	Learning and development	Testing different Linux distributions	Full Linux environment
Keeps your OS?	Yes	Yes	No (replaces OS)

3. Install Ubuntu Using WSL2 (Windows)

1. Open Windows PowerShell as Administrator.

```
wsl --install -d Ubuntu
```

2. Restart your computer if required.
3. Open Ubuntu from the Start menu.
4. Create a Linux username and password.
5. Verify the installation.

```
lsb_release -a
```

4. Install Linux on macOS or Linux

macOS (Using a VM)

- Use UTM (free) or VirtualBox.
- Download Ubuntu ISO from ubuntu.com.
- Create a new VM and install Ubuntu.



Linux Users

- You already have Linux.
- You can install additional distributions using a VM if needed (e.g., VirtualBox).
- Use your existing package manager (e.g., apt, dnf, pacman).



5. Update Packages

Keep your system up to date.

```
sudo apt update
```

6. Why Use sudo?

- The sudo command gives you temporary administrative privileges (root access).
- It allows you to install, update, and manage system software.
- Use it carefully and only when required.

7. Verify Your Installation

Check your Linux distribution:

```
cat /etc/os-release
```

Check your kernel version:

```
uname -r
```

8. Practical Exercise



1. Open your Linux terminal (WSL, VM, or native).
2. Run `cat /etc/os-release` and note your distribution.
3. Run `uname -r` and note your kernel version.
4. Run `sudo apt update` and confirm it completes without errors.
5. Take a screenshot of the output.

```
user@ubuntu:~$ cat /etc/os-release
NAME="Ubuntu"
VERSION="22.04.4 LTS (Jammy Jellyfish)"
ID=ubuntu
ID_LIKE=debian
PRETTY_NAME="Ubuntu 22.04.4 LTS"
user@ubuntu:~$ uname -r
5.15.0-100-generic
user@ubuntu:~$ sudo apt update
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease
Reading package lists... Done
```

9. Common Failure

- Virtualization is disabled in BIOS/UEFI.
- WSL2 fails to install or start.
- Error: "The distribution failed to start."
- Not enough system resources (RAM/CPU).
- Network issues when downloading packages.

Junior Engineer Tip

Start with WSL2 if you are using Windows. It is the fastest and safest way to learn Linux without replacing your main operating system.

Master the Linux Commands You Will Actually Use

1. Navigate and Check Your Location

Command	What It Does	Example
pwd	Shows your current working directory.	`pwd` → `/home/user`
ls	Lists files and directories.	`ls` → `app docs scripts`
cd	Change directory (navigate).	`cd docs` → `~/devops-homelab/docs`

2. Create Directories

Command	What It Does	Example
mkdir	Creates a new directory.	`mkdir myfolder` → `(no output)`
mkdir -p	Creates multiple directories at once.	`mkdir -p app/scripts` → `(no output)`

3. Create and Inspect Files

Command	What It Does	Example
touch	Creates an empty file.	`touch notes.txt` → `(no output)`
cat	Displays file content.	`cat notes.txt` → `Hello DevOps`
less	Views large files page by page.	`less longfile.log` → `(press q to exit)`

4. Copy and Move Files

Command	What It Does	Example
cp	Copies files or directories.	`cp notes.txt docs/` → `(no output)`
mv	Moves or renames files or directories.	`mv notes.txt docs/` → `(no output)`

5. Absolute vs Relative Paths

- **Absolute path:** Full path from the root (/).
  Example: `/home/user/devops-homelab/app`
- **Relative path:** Path from your current directory.
  Example: `../app` or `.../docs`
- Use `pwd` to confirm where you are before running commands to avoid mistakes.

6. Check Storage and Disk Usage

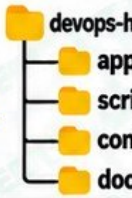
Command	What It Does	Example
df -h	Shows disk space usage (human-readable).	`Filesystem Size Used Avail` `/dev/sda1 100G 40G 60G`
du -sh	Shows the size of a file or directory.	`du -sh devops-homelab` → `120M devops-homelab`

7. Safe Command Habits

- ✓ Always check your current directory with `pwd`.
- ✓ Use `ls` before deleting or moving files.
- ✓ Be careful with `rm` (deletes files permanently).
- ✓ Use tab completion to avoid typing mistakes.
- ✓ Read the command and understand what it does.
- ✓ Start with non-destructive commands first.

8. Practical Exercise

1. Create a directory called `devops-homelab`.
2. Inside it, create `app`, `scripts`, `configs`, and `docs`.
3. Verify the structure using `ls` and `pwd`.




```
user@ubuntu:~$ mkdir -p devops-homelab/{app,scripts,configs,docs}
user@ubuntu:~$ cd devops-homelab
user@ubuntu:~/devops-homelab$ ls
app   configs   docs   scripts
```

9. Common Beginner Mistake

Executing commands in the wrong directory without first checking your location.

- You might create files in an unexpected place.
- You might delete the wrong files.
- You might move files and think they are lost.
- Always run `pwd` and `ls` before important commands.

Junior Engineer Tip

Always know where you are and what a command will do before you run it. A few seconds of checking can save you from losing important files or breaking your environment.

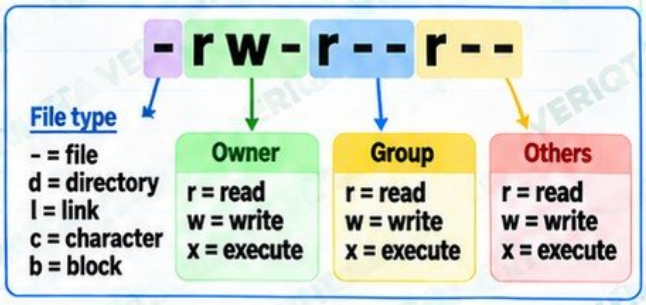
Users, Permissions, and Safe Administration

1. Users, Groups, and Ownership

- **Users:** A person or process that runs commands.
- **Groups:** Allow multiple users to share permissions.
- **Ownership:** Each file and directory has an owner (user) and a group.
- **Permissions:** Control who can read, write, or execute.



2. Understanding Permission Strings



3. Check Ownership and Permissions

Command	What It Does	Example Output
ls -l	Shows permissions, owner, group, size, and name.	`-rw-r--r-- 1 user user 1234 Oct 20 10:30 file.txt`
id	Shows your user and group information.	`uid=1000(user) gid=1000(user) groups=1000(user),4(adm), 24(cdrom)`

4. Common Permission Commands

Command	What It Does	Example
chmod	Changes permissions using numbers or symbols.	`chmod 600 file.txt` `chmod u+x script.sh`
chown	Changes file or directory owner.	chown user:group file.txt
chgrp	Changes the group owner.	chgrp devops file.txt
sudo	Runs a command with elevated privileges (administrator).	sudo apt update

5. Numeric Permission Examples

Number	Permission	Meaning
7	rwx	Read, write, execute
6	rw-	Read, write only
5	r-x	Read, execute only
4	r--	Read only
0	---	No permission

6. Why chmod 777 Is Dangerous

chmod 777 gives full access to everyone (owner, group, others).

- Anyone on the system can read, modify, or delete the file.
- It can expose sensitive data.
- It is a common security mistake in production.
- Use the minimum permission required.

Dangerous

```
chmod 777 file.txt
-rwxrwxrwx
```

(full access for everyone)

Better Example

```
chmod 600 file.txt
-rw-------
```

(only owner can read and write)

7. Protect SSH Keys and Configuration Files

- Store private keys in ~/.ssh/.
- Use chmod 600 for private keys.
- Use chmod 644 for public keys.
- Never share your private keys.
- Keep sensitive configuration files (e.g., .env, config) with restricted permissions.

```
mkdir -p  ~/.ssh
chmod 700 ~/.ssh
chmod 600 ~/.ssh/id_rsa
chmod 644 ~/.ssh/id_rsa_pub
```

8. Least Privilege in Everyday Work

- Use your normal user account for daily work.
- Use sudo only when required.
- Avoid running applications as root.
- Give the minimum permissions needed.
- This reduces security risks and prevents accidental damage.

9. Practical Exercise

1. Create a private directory: `mkdir ~/devops-homelab`
2. Set permissions: `chmod 700 ~/devops-homelab`
3. Create a test file: `touch ~/devops-homelab/test.txt`
4. Verify permissions: `ls -l ~/devops-homelab`
5. Try to read the file from another user (if available) to confirm access is restricted.

10. Production Connection

- Incorrect permissions can expose secrets (e.g., API keys, database credentials).
- Misconfigured permissions can prevent applications from starting.
- Following the principle of least privilege helps keep your systems secure and stable.

Common Beginner Mistake

Executing commands in the wrong directory without first checking your location using `pwd`.

Junior Engineer Tip

Always check the current directory (pwd) and think about the impact of each permission change before you run it.

Networking Fundamentals Inside Your Home Lab

1. Key Networking Concepts

- IP address: Identifies a device on a network.
- Loopback: 127.0.0.1 (points to your own machine).
- Private networks: Used inside home or local networks (e.g., 192.168.x.x, 10.x.x.x).
- Ports: Identify specific services on a device.
- These concepts help applications communicate.

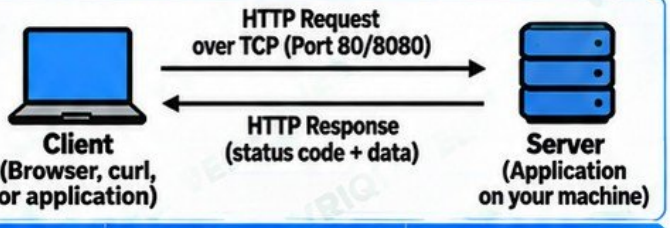
2. Common IP Address Types

Type	Example	Used For
Loopback	127.0.0.1	Your own machine (localhost)
Private (LAN)	192.168.1.10 10.0.0.5	Local/home networks (not accessible from Internet)
Public	203.0.113.5	Internet-facing (assigned by your ISP or cloud provider)

3. What Does Localhost Mean?

Environment	localhost Points To	Example
Your Laptop (Linux, Windows, macOS)	Your own machine	http://localhost:8080
Inside a Container	The container itself	http://localhost:8080
Inside a VM	The VM itself	http://localhost:8080
From another device on the network	Does NOT point to your machine	Uss your private IP (e.g., 192.168.1.10)

4. How Applications Communicate



Protocol	What It Does	Common Ports
TCP	Reliable communication	Any port (e.g., 80, 443)
HTTP	Web communication	80 (HTTP), 443 (HTTPS)
DNS	Converts domain names to IP	53

5. Check Network Interfaces

View your network interfaces and IP addresses.

```
user@ubuntu:~$ ip addr
1: lo: <LOOPBACK,UP,LOWER_UP>
    inet 127.0.0.1/8 scope host lo
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP>
    inet 192.168.1.10/24 brd 192.168.1.255
```

Look for:
- lo = loopback (127.0.0.1)
- eth0 or similar = local network (private IP)

6. Check Listening Ports

See which services are listening on your machine.

```
user@ubuntu:~$ ss -lnt
State   Recv-Q  Send-Q  Local Address:Port
LISTEN  0       4096    127.0.0.1:22
LISTEN  0       4096    0.0.0.0:8080
```

Common states:
- LISTEN = service is running and waiting for connections.
- 0.0.0.0 = available on all interfaces.

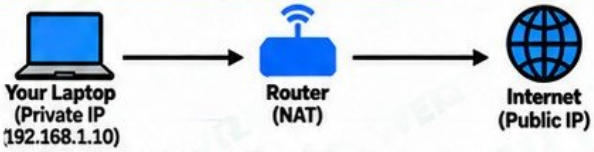
7. Test HTTP Connections with curl

Send an HTTP request to a local or remote service.

```
user@ubuntu:~$ curl http://localhost:8080
Hello from my app
user@ubuntu:~$ curl -I http://localhost:8080
HTTP/1.1 200 OK
Content-Type: text/plain
```

Useful options:
- -I = show headers only
- -v = verbose output
- -s = silent mode

8. Understand NAT and Firewall



- NAT (Network Address Translation) converts your private IP to a public IP for internet access.
- Firewalls can block incoming or outgoing traffic.

9. Common Connection Errors

Error Message	What It Means	How to Fix
Connection refused	No service is listening on that port.	Check if the application is running and the port is correct.
Connection timed out	The host did not respond.	Check network, firewall, or if the host is reachable.
Could not resolve host	DNS cannot find the domain name.	Check the domain name or DNS settings.

10. Practical Exercise

1. Start a simple web server: `python3 -m http.server 8080`
2. Check it is running: `ss -lnt | grep 8080`
3. Test from the same terminal: `curl http://localhost:8080`
4. Test from another terminal: `curl http://localhost:8080`
5. Stop the server: `Ctrl + C`

Troubleshooting Tip

A running process does not automatically mean its application is reachable. Always check the port, IP address, firewall, and network configuration.

Junior Engineer Tip

When troubleshooting, check each layer: application is running → port is open → network is accessible → firewall is not blocking → correct IP or domain is used.

Git and GitHub for Your Home Lab

1. What Is Version Control?

- Version control tracks changes to your files.
- You can see history, revert changes, and work with others.
- Infrastructure code, configuration files, and documentation should also be tracked.
- It helps you learn, experiment safely, and build a professional habit.



2. Configure Your Git Identity

```
user@ubuntu:~$ git config --global user.name "Your Name"
user@ubuntu:~$ git config --global user.email "youremail@example.com"
user@ubuntu:~$ git config --list
```

Why this matters:
- Your name and email are attached to your commits.
- It helps identify who made changes.
- Use the same identity across your machines.

3. Initialize a Repository

```
user@ubuntu:~$ mkdir devops-homelab
user@ubuntu:~$ cd devops-homelab
user@ubuntu:~/devops-homelab$ git init
Initialized empty Git repository in /home/user/devops-homelab/.git/
```

This creates a local Git repository in your project folder.

4. Stage and Commit Changes

```
user@ubuntu:~/devops-homelab$ git status
user@ubuntu:~/devops-homelab$ git add .
user@ubuntu:~/devops-homelab$ git commit -m "Initial commit: lab structure"
```

5. Understand .gitignore

- Use .gitignore to exclude files that should not be tracked.
- Common examples: .env, API keys, credentials, logs, and temporary files.

Example .gitignore

```
# Environment files
.env
# Logs
*.log
# Operating system files
.DS_Store
# IDE files
.vscode/
```

Why it is important:
- Prevents accidental exposure of sensitive data.
- Keeps your repository clean.
- Avoids committing large or unnecessary files.

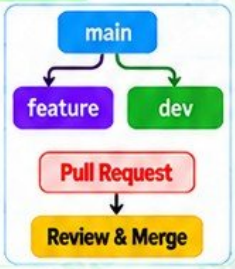
6. Connect to GitHub

1. Create a new repository on GitHub (empty).
2. Copy the repository URL.
3. Add the remote and push your code.

```
user@ubuntu:~/devops-homelab$ git remote add origin https://github.com/your-username/devops-homelab.git
user@ubuntu:~/devops-homelab$ git branch -M main
user@ubuntu:~/devops-homelab$ git push -u origin main
```

7. Branches, Pull Requests, and Code Review

- Branches let you work on new features without breaking your main code.
- Pull requests are used to review and merge changes.
- Code review helps catch mistakes and improve quality.
- Common branch names: main, dev, feature-<name>.



8. Local vs Remote Repository

Local Repository	Remote Repository (GitHub)
Stored on your computer	Stored on GitHub servers
Works offline	Accessible from anywhere
Keeps local commit history	Central place to share and backup code
Useful for development and testing	Useful for collaboration, backup, and deployment

9. Practical Exercise

1. Create a folder structure: devops-homelab.
2. Add folders: app, scripts, configs, docs.
3. Create a README.md file with a short description.
4. Initialize a Git repository, commit the files, and push to GitHub.
5. Verify that your files appear on GitHub.

Beginner Mistake
- Accidentally committing API keys, credentials, or .env files.
- Forgetting to use .gitignore.
- Committing large files such as logs or system files.
- Using the wrong branch and pushing to main by mistake.
- Not checking the files before committing (use git status).

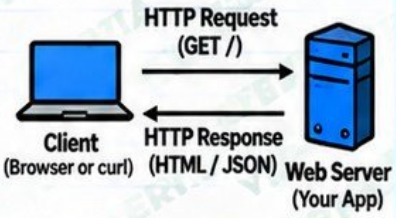
Junior Engineer Tip

Make small, clear commits with meaningful messages. This helps you and others understand what changed.

Build Your First Application in the Lab

1. What Is a Web Application?

- A web application is a program that runs on a server and responds to HTTP requests.
- A web server listens for requests and returns responses.
- You access it using a browser or a tool like curl.
- This application will run on your laptop inside your lab environment.



2. Project Structure

```
homelab-app/
├── app.py              # application code
├── requirements.txt    # dependencies
└── venv/               # virtual environment
                        (created later)
```

3. Create a Python Virtual Environment

1. Create a project folder and move into it.
2. Create a virtual environment.
3. Activate the virtual environment.
4. Install required packages.

```
user@ubuntu:~$ mkdir homelab-app
user@ubuntu:~$ cd homelab-app
user@ubuntu:~/homelab-app$ python3 -m venv venv
user@ubuntu:~/homelab-app$ source venv/bin/activate
(venv) user@ubuntu:~/homelab-app$ pip install flask
```

4. Write a Simple Python Application

app.py

```
from flask import Flask, jsonify
app = Flask(__name__)
@app.route("/")
def home():
    return "<h1>Welcome to homelab-app</h1>"
@app.route("/health")
def health():
    return jsonify({"status": "ok"}), 200
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
```

5. Application Dependencies

- Dependencies are external packages that your application needs.
- In this example, we use Flask.
- They are listed in requirements.txt.
- A virtual environment keeps them isolated from your system Python.

requirements.txt

```
Flask==3.0.0
```

Why use a virtual environment?

- Keeps dependencies isolated.
- Prevents version conflicts.
- Makes your project portable.

6. Run the Application

1. Make sure you are in the project folder and the virtual environment is active.
2. Start the application.
3. Keep the terminal open while it runs.

```
(venv) user@ubuntu:~/homelab-app$ python app.py
 * Running on http://0.0.0.0:8080
 * Press CTRL+C to quit
```

7. Test the Application with curl

```
# Test the home page
user@ubuntu:~$ curl http://localhost:8080/
<h1>Welcome to homelab-app</h1>
# Test the health endpoint
user@ubuntu:~$ curl http://localhost:8080/health
{"status": "ok"}
```

Expected Response

/

```
<h1>Welcome
    to homelab-app</h1>
```

/health

```
{"status": "ok"}
```

8. Why Health Endpoints Are Useful

- They confirm that the application is running.
- Used by monitoring tools and load balancers.
- Help detect failures automatically.
- Should return a simple and fast response (e.g., 200 OK).

9. Practical Exercise



1. Create a folder called homelab-app.
2. Set up a Python virtual environment.
3. Install Flask and create app.py.
4. Run the application and test both / and /health.
5. Confirm that the health endpoint returns a successful response (status: ok).

10. Production Connection



- Applications need to be observable before they can be operated reliably.
- Health endpoints, logs, and metrics help engineers detect and troubleshoot issues.
- A simple application is the first step before deployment automation.

Beginner Mistake

- Forgetting to activate the virtual environment.
- Not installing the required dependencies.
- Application runs but testing the wrong port.
- Assuming the application is reachable from other devices without checking the network configuration.

Junior Engineer Tip

Start with a simple application. Once it works, you can extend it with a database, Docker, and deployment tools in later labs.

Install Docker and Understand Containers

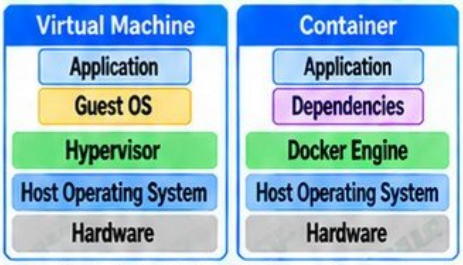
1. Why Use Containers?

- Containers package your application and its dependencies into a portable unit.
- They run the same way in development, testing, and production.
- They help you avoid "it works on my machine" problems.
- DevOps engineers use containers to build, test, and deploy applications reliably.

2. Key Docker Concepts

Concept	What It Means
Docker Engine	Runs and manages containers.
Image	A read-only template used to create containers.
Container	A running instance of an image.
Registry	Stores and distributes images (e.g., Docker Hub).

3. Container vs Virtual Machine



- VMs include a full operating system.
- Containers share the host operating system.
- Containers are lighter and start faster.

4. Install Docker

On Ubuntu (Linux)

```
sudo apt update
sudo apt install -y
docker.io
sudo systemctl enable
 docker
sudo systemctl start docker
```

On Windows

- Install Docker Desktop from the official website.
- Enable WSL2.
- Follow the installation wizard.

On macOS

- Install Docker Desktop from the official website.
- Follow the installation wizard.
- Docker runs natively on macOS.

5. Verify Installation

```
user@ubuntu:~$ docker version
Client: Docker Engine - Community
 Version:           27.0.1
 API version:       1.46
 Go version:        go1.22.0
 ...
```

If you see version information, Docker is installed correctly.

6. Run a Test Container

```
user@ubuntu:~$ docker run hello-world
Hello from Docker!
This message shows that your installation
appears to be working correctly.
```

7. Inspect Containers

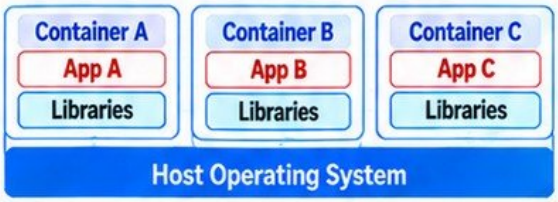
```
# List running containers
user@ubuntu:~$ docker ps
# List all containers (including stopped)
user@ubuntu:~$ docker ps -a
# View details of a container
user@ubuntu:~$ docker inspect <container_id>
```

8. Common Docker Commands

Command	What It Does
`docker pull <image>`	Download an image.
`docker run <image>`	Run a container.
`docker ps`	List running containers.
`docker ps -a`	List all containers.
`docker stop <id>`	Stop a container.
`docker start <id>`	Start a container.
`docker rm <id>`	Remove a container.
`docker images`	List downloaded images.

9. Container Isolation and Resource Usage

- Containers share the host operating system kernel.
- They are isolated using namespaces and cgroups.
- You can set resource limits (CPU, memory) to prevent a container from using all system resources.



10. Security Considerations

- Running Docker usually requires root or sudo privileges.
- Unrestricted Docker access can allow a container to access your host system.
- Only run trusted images.
- Avoid mounting sensitive directories.
- Use least privilege and limit container permissions.

11. Practical Exercise

1. Run the official test image:
   `docker run hello-world`
2. List all containers:
   `docker ps -a`
3. Start a real container:
   `docker run -d --name nginx-test -p 8080:80 nginx`
4. Verify it is running:
   `docker ps`
5. Open in browser or test with `curl`:
   `curl http://localhost:8080`
6. Stop and remove the container:
   `docker stop nginx-test`
   `docker rm nginx-test`

Beginner Mistake

- Running untrusted images from unknown sources.
- Giving containers full access to your host system.
- Assuming a container is secure without proper configuration.
- Not removing unused containers and images (wastes disk space).

Junior Engineer Tip

Start with simple containers. Understand how they work, then gradually experiment with more complex setups. A small working container that you understand is more valuable than a complex environment you cannot troubleshoot.

Containerize Your Own Application

1. Why containerize your application?

- Packages your application and dependencies into a single unit.
- Runs the same way in development, testing, and production.
- Makes deployment faster and more reliable.
- Helps avoid "it works on my machine" problems.

2. Project Structure

```
homelab-app/
├── app.py              # application code
├── requirements.txt    # dependencies
├── Dockerfile          # container instructions
└── .dockerignore       # files to exclude
```

3. Dockerfile Explained

Instruction	What It Does
FROM	Sets the base image (e.g., python:3.11).
WORKDIR	Sets the working directory inside the container.
COPY	Copies files from your project to the image.
RUN	Runs a command during image build (e.g., install packages).
USER	Sets a non-root user for security.
CMD	Command to run when the container starts.

4. Example Dockerfile (Python App)

```
FROM python:3.11-slim
WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY app.py .

# Create a non-root user
RUN useradd -m appuser
USER appuser

EXPOSE 8080
CMD ["python", "app.py"]
```

5. .dockerignore File

```
__pycache__/
.venv/
.env
.git/
*.pyc
*.log
node_modules/
.DS_Store
```

Why use it?

- Keeps the image small.
- Avoids copying sensitive files.
- Speeds up the build process.

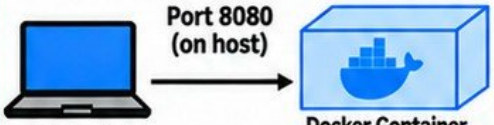
6. Build the Docker Image

```
user@ubuntu:~/homelab-app$ docker build -t homelab-app:1.0 .
Sending build context to Docker daemon...
Step 1/6 : FROM python:3.11-slim
...
Successfully built homelab-app:1.0
```

7. Run the Container

```
user@ubuntu:~/homelab-app$ docker run -d \\
  -p 8080:8080 \
  --name homelab-app \
  homelab-app:1.0
a1b2c3d4e5f6... (container ID)
```

8. Application Networking and Port Publishing



- -p 8080:8080 maps the host port to the container port.
- You can access the application using http://localhost:8080.
- Containers use an internal network to communicate.
- You can create custom networks later for multi-container apps.

9. Test the Running Application

```
# Test the home page
user@ubuntu:~$ curl http://localhost:8080/
<h1>Welcome to homelab-app</h1>

# Test the health endpoint
user@ubuntu:~$ curl http://localhost:8080/health
{"status":"ok"}
```

10. Key Best Practices

- Use an official and minimal base image.
- Use .dockerignore to exclude unnecessary files.
- Do not embed secrets (API keys, credentials, .env files).
- Run containers as a non-root user.
- Keep images small and focused.
- Use specific image tags (e.g., homelab-app:1.0).
- Document how to build and run the container.

11. Practical Exercise

1. Build the Docker image: docker build -t homelab-app:1.0
2. Run the container: docker run -d -p 8080:8080
3. Test the application: curl http://localhost:8080/
4. Test the health endpoint: curl http://localhost:8080/health
5. Inspect the running container: docker ps
6. Stop and remove the container: docker stop homelab-app
   docker rm homelab-app

Common Failure

- Container starts but the application cannot be reached.
- Application is listening on the wrong interface (use 0.0.0.0).
- Using the wrong port mapping (host:container).
- Firewall or port is blocked.
- Application exits immediately (check logs with docker logs).

Junior Engineer Tip

Use non-root containers, keep images small, and always test that your application is reachable from outside the container. A working health endpoint is a good sign that your container is ready for deployment.

Add a Database and Persistent Storage

1. Why Application Data Needs Care

- Application data is important and must not be lost.
- Some services are stateless (e.g., web apps) and some are stateful (e.g., databases).
- Databases store persistent data that must survive container restarts.
- Data needs backups, access controls, and proper storage management.

2. Stateless vs Stateful Services

Type	Characteristics	Examples
Stateless	Does not store persistent data. Can be scaled and replaced easily.	Web application, API server
Stateful	Stores persistent data that must be retained.	Database, message queue, file storage

3. Meet PostgreSQL

- PostgreSQL is a powerful, open source relational database.
- It is commonly used in production.
- It stores data in tables and supports transactions, indexes, and backups.

PostgreSQL

4. Docker Volumes and Bind Mounts

Docker Volume (recommended for databases)

- Managed by Docker.
- Data stored outside the container.
- Survives container deletion.
- Best for databases.

```
docker volume create pgdata
docker run -d \
  -v pgdata:/var/lib/postgresql/data \
  postgres:16
```

Bind Mount (useful for development files)

- Maps a host folder to a container folder.
- You control the location.
- Good for application code, configs, and logs.

```
docker run -d \
  -v $(pwd)/app:/app \
  myapp:1.0
```

5. Docker Compose Example (App + PostgreSQL)

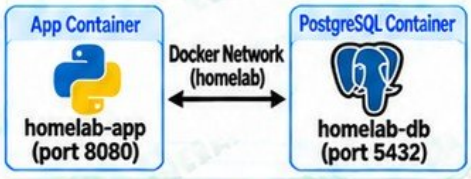
docker-compose.yml

```yaml
version: '3.9'
services:
  db:
    image: postgres:16
    container_name: homelab-db
    environment:
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: strongpassword
      POSTGRES_DB: homelabdb
    volumes:
      - pgdata:/var/lib/postgresql/data
    networks:
      - homelab
    healthcheck:
      test: ["CMD-SHELL", 'pg_isready -U appuser"]
      interval: 5s
      retries: 5
  app:
    build: .
    container_name: homelab-app
    ports:
      - "8080:8080"
    environment:
      DATABASE_URL: postgresql://appuser:strongpassword@db:5432/homelabdb
    depends_on:
      db:
        condition: service_healthy
    networks:
      - homelab

volumes:
  pgdata:
networks:
  homelab:
```

6. Container Networking

- Containers can communicate using a Docker network.
- In the example, the app connects to the database using the service name "db".
- They do not need to use localhost.
- A custom network keeps the services isolated.



7. Use Environment Variables for Credentials

- Do not hardcode database credentials in your code.
- Use environment variables or a `.env` file.
- Keep the `.env` file out of version control using `.gitignore`.

Example .env file (do not commit)

```
POSTGRES_USER=appuser
POSTGRES_PASSWORD=strongpassword
POSTGRES_DB=homelabdb
DATABASE_URL=postgresql://appuser:strongpassword@db:5432/homelabdb
```

8. Start the Services

```
user@ubuntu:~/homelab-app$ docker compose up -d
Creating network "homelab-app_homelab"
Creating volume "homelab-app_pgdata"
Creating homelab-db  ... done
Creating homelab-app ... done
```

9. Verify the Services

```
# Check running containers
user@ubuntu:~$ docker compose ps
NAME          COMMAND              STATE     PORTS
homelab-db    docker-entrypoint    running   5432/tcp
homelab-app   python app.py        running   0.0.0.0:8080->8080/tcp
# Test the application
user@ubuntu:~$ curl http://localhost:8080/health
{"status": "ok"}
```

10. Data Persistence Test

```
# Stop and remove containers
docker compose down
# Start them again
docker compose up -d
# Verify the data is still there
docker exec -it homelab-db \
  psql -U appuser -d homelabdb \
  -c "\dt"
```

Common Failure

- Container starts but the application cannot be reached.
- Wrong interface or port configuration.
- Forgetting to use a persistent volume.
- Database not ready when the application starts.
- Committing database credentials to the repository.

Production Awareness

- A persistent volume is not a backup.
- Data must also be backed up regularly.
- Plan for disaster recovery.
- Protect database credentials and restrict access.
- Use monitoring to detect issues early.

Junior Engineer Tip

Use Docker Compose for multi-container applications. It makes your setup easy to reproduce, portable, and closer to real-world environments.

Create Your First CI Pipeline With GitHub Actions

1. What Is Continuous Integration?

- Continuous Integration (CI) automatically validates code changes every time you push to the repository.
- It runs tests, checks for errors, and helps catch problems early.
- CI reduces manual work and prevents broken code from being deployed.
- It is a key practice in DevOps and modern software development.

2. Key GitHub Actions Concepts

Term	What It Means
Workflow	A YAML file that defines automation steps.
Job	A set of steps that run on a runner.
Runner	A server provided by GitHub to run your jobs.
Step	A single command or action in a job.
Trigger	An event that starts the workflow (e.g., push, pull request).

3. CI Workflow Flow



4. Create the Workflow File

1. Create a `.github/workflows` directory in your repository.
2. Add a file named `ci.yml`.
3. Copy the example workflow below.

.github/workflows/ci.yml

```yaml
name: CI - Homelab App
on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
    - name: Checkout code
      uses: actions/checkout@v4
    - name: Set up Python
      uses: actions/setup-python@v5
      with:
        python-version: '3.11'
    - name: Install dependencies
      run: pip install -r requirements.txt
    - name: Run tests
      run: python -m pytest -v
```

5. What the Workflow Does

1. Triggers on push and pull requests to the main branch.
2. Checks out your code.
3. Sets up the Python environment.
4. Installs application dependencies.
5. Runs your tests.
6. Fails the workflow if any test fails (non-zero exit code).



6. Install Dependencies and Run Tests

```
# Install dependencies
pip install -r requirements.txt

# Run a simple test (example with pytest)
pytest -v

# Test a health endpoint (if using a running app)
curl http://localhost:8000/health
# Expected response
{"status": "ok"}
```

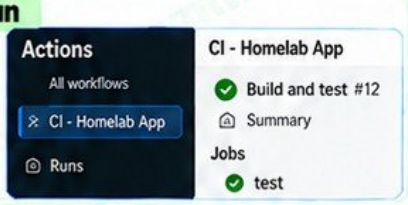
Example Test File (tests/test_app.py)

```python
from app import app
import pytest

def test_health_endpoint():
    client = app.test_client()
    response = client.get("/health")
    assert response.status_code == 200
    assert response.json["status"] == "ok"
```

7. Push a Change and Check the Run

1. Commit and push your code to GitHub.
2. Go to the Actions tab in your repository.
3. Open the latest workflow run.
4. Check the job status and logs.
5. You should see a green check if tests passed.



8. Intentionally Trigger a Failing Test

1. Change the expected status code to a wrong value (e.g., 404) in your test file.
2. Commit and push the change.
3. Check the Actions run again.
4. The workflow should show a red failure with error logs.

Failed Test Output (Example)

```
> assert 404 == 200
E  assert 404 == 200
tests/test_app.py:8: AssertionError
```

9. Important Security Notes

- Do not expose credentials or API keys. in your workflow file.
- Use GitHub Secrets for sensitive data.
- Be careful with workflows from untrusted sources (e.g., pull requests from forks).
- Use least privilege for workflow permissions.

10. Practical Exercise

1. Create `.github/workflows/ci.yml`.
2. Push a change and verify the workflow runs.
3. Check the tests pass with a green check.
4. Modify a test to fail and confirm the workflow shows a red failure with logs.

Junior Engineer Tip

A green pipeline proves that the configured checks passed, not that the application is free of defects. You still need code reviews, proper testing, and real-world monitoring.

Why This Matters

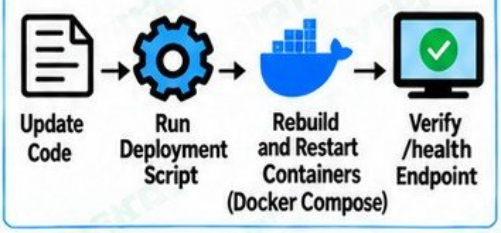
- Automated tests catch issues early.
- Prevents broken code from reaching production.
- Builds confidence in your deployments.
- Saves time and reduces risk.

Automate Your Local Deployment

1. What Is Deployment Automation?

- Turns a manual deployment into a controlled and repeatable process.
- Reduces human error and saves time.
- Ensures consistent deployments.
- Helps you detect failures early.
- A key practice used in real DevOps environments.

2. Deployment Flow



3. Project Structure

```
homelab-app/
├── app/          # application code
├── scripts/      # deployment script
├── docker-compose.yml
└── .env          # environment variables
```

4. Example Deployment Script (deploy.sh)

scripts/deploy.sh

```
#!/bin/bash
set -euo pipefail

echo "Starting deployment..."

echo "1. Pull latest code (if using Git)"
# git pull

echo "2. Build and start containers"
docker compose build
docker compose up -d

echo "3. Wait for application to be ready..."
sleep 5

echo "4. Check health endpoint"
HTTP_CODE=$(curl -s -o /dev/null -w "%{http;?0de}" \
                http://localhost:8080/health)
if [ "$HTTP_CODE" = "200"]; then
  echo "✅ Deployment successful! (HTTP $HTTP_CODE")
else
  echo "❌ Health check failed! (HTTP $HTTP_CODE")
  exit 1
fi
```

5. Make the Script Executable

```
user@ubuntu:~/homelab-app$ chmod +x scripts/deploy.sh
```

6. Run the Deployment

```
user@ubuntu:~/homelab-app$ ./scripts/deploy.sh
Starting deployment...
1. Pull latest code (if using Git)
2. Build and start containers
...
✅ Deployment successful! (HTTP 200)
```

7. Verify the Running Version

```
# Check container status
docker compose ps

# Test the health endpoint
curl http://localhost:8080/health
{"status": "ok", "version": "1.2.0"}
```

Expected Response

```
{
  "status": "ok",
  "version": "1.2.0"
}
```

8. Important Concepts

Concept	What It Means
Idempotency	Running the script multiple times produces the same result.
Repeatability	The deployment process is consistent and reliable.
Exit Codes	Commands return a code (0 = success, non-zero = failure).
Error Handling	The script should stop if a command fails (set -e).
Rollback	Keep previous images or tags to restore if the new version fails.
Versioned Images	Use image tags like 1.0, 1.1, 1.2 instead of only latest.

9. Example docker-compose.yml

docker-compose.yml

```
version: '3.9'
services:
  app:
    build: .
    image: homelab-app:1.2.0
    ports:
      - "8080:8080"
    env_file:
      - .env
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:16
    container_name: homelab-db
    environment:
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: strongpassword
      POSTGRES_DB: homelabdb
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U appuser"]
      interval: 5s
      retries: 5
volumes:
  pgdata:
```

10. Practical Exercise

1. Update your application code (e.g., change the version or a message).
2. Run the deployment script: `./scripts/deploy.sh`
3. Check that containers are rebuilt and restarted.
4. Verify the /health endpoint returns 200 and the new version.
5. Run the script again to confirm it is repeatable (idempotent).
6. Intentionally make a change that causes a failure and confirm the script stops with an error.



Common Failure

- The deployment command completes, but the new application fails its health check.
- The script ignores errors and continues.
- The application listens on the wrong port or interface.
- A container starts with the old image insstead of the new one.

Junior Engineer Tip

- Always check the health endpoint after deployment.
- Do not ignore errors (use set -e).
- Use versioned images for rollback.
- Keep your deployment script simple and easy to understand.
- Test failure scenarios to build confidence.

Production Awareness

- Automate deployments to reduce human error.
- Have a rollback plan (previous image tag).
- Monitor the application after deployment.
- A successful deployment script does not guarantee the application is fully working.
- Add logging, alerts, and monitoring in real environments.

Monitor Your Lab With Metrics and Logs

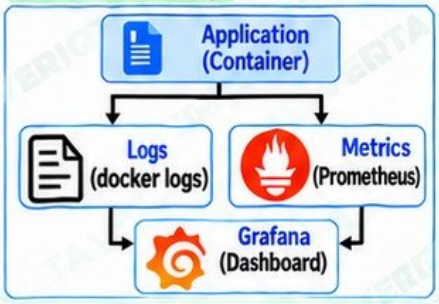
1. Why Monitoring Matters?

- Helps you know if your application is healthy.
- Helps you find problems early.
- Reduces downtime and improves reliability.
- Gives visibility into performance, errors, and resource usage.
- A key skill for DevOps and modern engineering roles.

2. Logs, Metrics, and Traces

Type	What It Is	Example
Logs	Text records of what happened.	Application logs, error messages.
Metrics	Numeric data collected over time.	CPU, memory, request count.
Traces	Track a request across multiple services.	Used in distributed systems to follow a request path.

3. Monitoring Architecture (Your Lab)



4. View Container Logs

```
user@ubuntu:~$ docker logs homelab-app
INFO:   Starting application...
INFO:   Connected to database.
INFO:   Server running on port 8080
GET  /  200 OK
GET  /health  200 OK
ERROR:  Example error message
```

5. View Container Resource Usage

```
user@ubuntu:~$ docker stats
CONTAINER ID   NAME          CPU %    MEM USAGE
a1b2c3d4e5f6   homelab-app   1.23%    120MiB / 1GiB
b2c3d4e5f6g7   homelab-db    0.45%    80MiB / 1GiB
```

What to Look For?

- CPU usage spikes.
- High memory usage.
- Container restarting frequently.
- Growing disk usage (logs or data).

6. Install Monitoring Tools

1. Use Docker Compose to run Prometheus and Grafana.
2. Prometheus collects metrics from your containers.
3. Grafana visualizes the metrics in dashboards.
4. Both run as containers in your lab.

7. Example docker-compose.yml (Prometheus + Grafana)

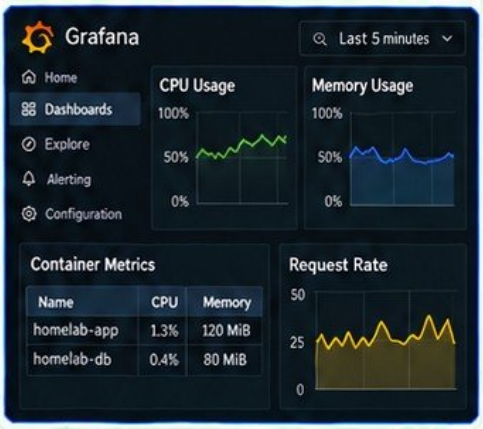
docker-compose.yml

```
version: '3.9'
services:
  prometheus:
    image: prom/prometheus:latest
    ports:
      - "9090:9090"
    volumes:
      - ./prometheus.yml:/etc/prometheus/
        prometheus.yml
  grafana:
    image: grafana/grafana:latest
    ports:
      - "3000:3000"
```

8. Access the Dashboards

- Prometheus: http://localhost:9090
- Grafana: http://localhost:3000
- Default Grafana login:
  - Username: admin
  - Password: admin
- Add Prometheus as a data source in Grafana and import a dashboard (e.g., Node Exporter or Docker dashboard).

9. Example Grafana Dashboard



10. Useful Prometheus Metrics (Examples)

Metric	What It Shows
container_cpu_usage_seconds_total	CPU usage of containers.
container_memory_usage_bytes	Memory usage.
container_fs_usage_bytes	Disk usage.
up	Whether a target is reachable.
http_requests_total	Number of HTTP requests.
http_request_duration_seconds	Request response time.

11. Create a Simple Alert (Concept)

- An alert should detect actual problems, not normal activity.
- Examples of useful alerts:
  - Container is down.
  - High CPU or memory usage.
  - Application health endpoint failing.
  - Disk space is nearly full.
  - Avoid too many alerts, especially on a small laptop.

12. Practical Exercise

1. Run your application and generate some logs (e.g., make a few requests).
2. View logs using `docker logs homelab-app`
3. Check resource usage using `docker stats`
4. (Optional) Run Prometheus and Grafana with Docker Compose if your computer has enough resources.
5. Open Grafana and view ayp dashboard.
6. Make a change to your app and observe the metrics and logs.

Common Failure

- Application is running but logs show errors.
- High resource usage causes slow performance.
- Monitoring tools are installed but not properly configured.
- Too many alerts make important issues easy to miss.
- No alerting for real failures.

Junior Engineer Tip

- Logs tell you what happened.
- Metrics show you what is happening now.
- Monitoring should help you find problems before users report them.
- Start with simple dashboards and a few important alerts. Keep it lightweight for a local lab.

Production Connection

- In production, monitoring helps engineers recognize and fix problems early.
- It improves reliability, reduces downtime, and builds trust with users.
- Good monitoring is essential for operating real-world applications.

Troubleshoot Five Real Home Lab Failures

1. Troubleshooting Approach

- Use evidence, not guesses.
- Reproduce the problem.
- Check logs, error messages, and system state.
- Identify the root cause.
- Apply a fix and verify the solution.
- Document what you learned.

2. Useful Troubleshooting Commands

```
docker ps              # running containers
docker ps -a           # all containers
docker logs <name>     # view container logs
docker inspect <name>  # detailed information
docker stats           # resource usage
ss -lntp               # check listening ports
lsof -i :<port>        # find process using a port
df -h                  # disk space usage
du -sh <folder>        # check folder size
docker system df       # Docker disk usage
```

3. Troubleshooting Steps

1. Identify the problem
2. Collect evidence
3. Find the root cause
4. Apply a fix
5. Verify recovery

4. Failure 1: Port Already in Use

Error: Port 8080 is already in use.

```
$ docker run -p 8080:8080 myapp
Error: bind: addrees already in use
```

Find the process

```
$ ss -lntp | grep :8080
$ lsof -i :8080
```

Example output:

```
COMMAND   PID  USER  FD  TYPE DEVICE SIZE/OFF NODE NAME
python    1234 user  3u  IPv4 12345    0t0   TCP *:8080
```

Fix

```
$ kill 1234    # stop the process
# or change to a different port
```

5. Failure 2: Container Exits Immediately

Symptom: Container starts and exits.

```
$ docker ps -a
CONTAINER ID   IMAGE    STATUS
a1b2c3d4       myapp    Exited (1) 5 seconds ago
```

Check logs and exit status

```
$ docker logs a1b2c3d4
$ docker inspect a1b2c3d4 --format
 '{{.State.ExitCode}}'
```

Fix

- Read the error message in the logs.
- Fix the configuration or code.
- Rebuild the image and run again.

6. Failure 3: Permission Denied

Error: Permission denied when accessing a file.

```
$ cat /data/app.log
cat: /data/app.log: Permission denied
```

Check file ownership and permissions

```
$ ls -l /data/app.log
$ ls -ld /data
```

Example output:

```
-rw------- 1 root root 1024 Oct 20 10:00 /data/app.log
```

Fix

- Use the correct user or group.
- Change ownership: `sudo chown user:user /data/app.log`
- Or adjust permissions: `sudo chmod 644 /data/app.log`

7. Failure 4: Application Cannot Reach PostgreSQL

Error: Connection to database fails.

```
$ docker logs homelab-app
ERROR: could not connect to server: Connection refused
Is the server running on host "postgres" (172.19.0.3)
  and accepting TCP/IP connections on port 5432?
```

Check the common causes

- Correct service name (use "postgres", not localhost).
- Both containers are on the same Docker network.
- PostgreSQL is running and ready.
- Credentials and environment variables are correct.
- Check database logs: `docker logs homelab-db`

Fix

- Use the correct service name in your configuration.
- Ensure Docker Compose services are on the same network.
- Wait for database readiness (use depends_on or a health check).
- Verify credentials and try again.

8. Failure 5: Disk Space Running Low

Symptom: No space left on device.

```
$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda1        50G   48G  1.2G  97% /
```

Find large files and Docker usage

```
$ du -sh /* 2>/dev/null | sort -h
$ docker system df
$ docker system df -v
```

Example output:

```
TYPE          TOTAL  ACTIVE  SIZE    RECLAIMABLE
Images        10     5       2.8GB   1.5GB
Containers    5      2       1.2GB   800MB
Volumes       3      3       5.0GB   4.8GB
```

Fix (cleanup safely)

- Remove unused containers: `docker container prune`
- Remove unused images: `docker image prune`
- Remove unused volumes: `docker volume prune`
- Remove everything not used: `docker system prune -a`

9. Verification After Fix

- The application starts and stays running.
- No more error messages in logs.
- The service is reachable (e.g., `curl http://localhost:8080/health`).
- Database connections work.
- Disk space has been reclaimed.

10. Practical Exercise

1. Intentionally break one non-production component (e.g., stop the database or change a port).
2. Collect evidence using the commands above.
3. Identify the root cause.
4. Apply a fix.
5. Verify that the service is working again.
6. Write a short note summarizing what happened and what you learned.

Common Failure

- Fixing the symptom instead of the root cause.
- Ignoring log messages.
- Not checking exit codes.
- Running out of disk space.
- Changing multiple settings at once.
- Not verifying that the issue is actually resolved.

Junior Engineer Tip

- Diagnose before changing settings.
- Use logs, metrics, and system state as evidence.
- Make one change at a time.
- Random fixes can hide the real cause.
- Document the problem and the solution.

Production Connection

- A clear troubleshooting process reduces downtime.
- Real incidents look similar but can be more complex.
- Good observability (logs, metrics, alerts) makes troubleshooting faster.
- The same skills you use in a home lab apply in in production environments.

Run Kubernetes Locally Without Paying for a Cluster

1. Why Kubernetes Exists?

- Runs and manages containers at scale.
- Handles application deployment, scaling, and self-healing.
- Keeps applications highly available.
- Manages networking, storage, and configuration.
- Widely used in production environments.

2. Key Kubernetes Concepts

Concept	What It Means
Cluster	A group of machines (nodes) that run your applications.
Node	A machine that runs Pods.
Pod	The smallest unit. Runs one or more containers.
Deployment	Manages and keeps your Pods running.
Service	Provides a stable network endpoint to access Pods.

3. Docker Compose vs Kubernetes

Docker Compose	Kubernetes
For small, simple setups on one machine.	Designed for larger, distributed environments.
Uses a docker-compose.yml file.	Uses YAML manifests (e.g., Deployment, Service).
Manages multiple containers.	Manages containers with scaling, self-healing, and more features.
Good for local development and simple apps.	Used in production and complex environments.

4. Local Kubernetes Options

kind (Kubernetes in Docker)

- Runs Kubernetes in Docker containers.
- Lightweight and easy to set up.
- Good for learning and development.

minikube

- Runs a single-node Kubernetes cluster on your machine.
- Supports multiple drivers (Docker, VirtualBox).
- Good for learning and testing.

5. Local Environment Requirements

- 64-bit operating system (Linux, macOS, or Windows).
- At least 4 CPU cores (recommended).
- Minimum 8 GB RAM (recommended).
- At least 20 GB of free disk space.
- Docker installed (for kind).
- kubectl installed.

6. Create a Local Cluster (kind)

Create a cluster

```
# Create a kind cluster
kind create cluster --name homelab

# Check cluster nodes
kubectl get nodes
```

Verify cluster

```
kubectl cluster-info
kubectl get nodes
```

7. Deploy homelab-app to Kubernetes

1. Create Deployment (deployment.yaml)

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: homelab-app
spec:
  replicas: 1
  selector:
    matchLabels:
      app: homelab-app
  template:
    metadata:
      labels:
        app: homelab-app
    spec:
      containers:
      - name: homelab-app
        image: homelab-app:1.0
        ports:
        - containerPort: 8080
```

2. Create Service (service.yaml)

```
apiVersion: v1
kind: Service
metadata:
  name: homelab-app
spec:
  type: NodePort
  selector:
    app: homelab-app
  ports:
  - port: 80
    targetPort: 8080
    nodePort: 30080
```

3. Apply the manifests

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

8. Inspect the Deployment

```
# View pods
kubectl get pods

# View detailed information
kubectl describe pod <pod-name>

# View services
kubectl get services

# Access the application
# (using NodePort, e.g. 30080)
curl http://localhost:30080
```

9. Stop and Delete the Cluster

```
# Delete the cluster when done
kind delete cluster --name homelab

# Verify it is removed
kubectl get nodes
```

10. Practical Exercise

1. Create a local Kubernetes cluster (using kind or minikube).
2. Deploy the homelab-app using the YAML files above.
3. Check that the Pod is running.
4. Access the application using the NodePort.
5. Inspect the Pod, Service, and logs.
6. Delete the cluster when finished.

11. Common Failure: Pod Pending

- Not enough CPU or memory on your laptop.
- Docker is not running (for kind).
- Incorrect container image name.
- Image pull errors (no internet or private image).
- Resource requests are too high.
- Node is NotReady.
- Storage or networking issues.

Example: Pod Pending (Output)

```
$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
homelab-app-6d9f7d4	0/1	Pending	0	2m

Junior Engineer Tip

- Check the node resources (kubectl describe node).
- Use realistic resource requests and limits.
- Read events (kubectl describe pod) for the real cause.
- Do not randomly change settings.
- Fix the underlying problem and verify the solution.

Production Connection. Kubernetes helps engineers run, scale, and recover applications reliably.
The same concepts you learn locally are used in real cloud environments.
Understanding Pods, Deployments, and Services prepares you for production systems.

Learn Infrastructure as Code With Terraform

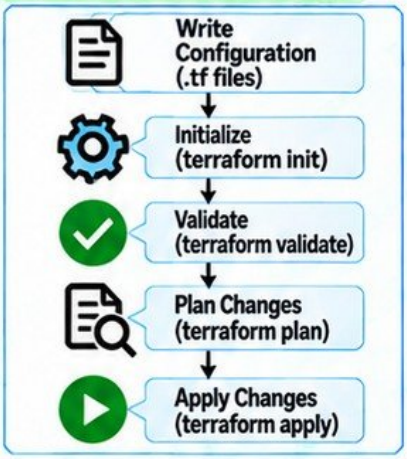
1. What Is Infrastructure as Code?

- Infrastructure as Code (IaC) means defining and managing infrastructure using code instead of manual steps.
- Infrastructure can be version controlled, reviewed, and repeated.
- Reduces human error and improves consistency.
- Widely used in modern DevOps and cloud environments.

2. Key Terraform Concepts

Concept	What It Means
Provider	Plugin that allows Terraform to manage a platform (e.g., local, AWS, Azure).
Resource	The infrastructure object you want to create or manage (e.g., a file, server, or bucket).
Configuration	Written in HCL (HashiCorp Configuration Language).
State	A file that keeps track of the real infrastructure.

3. Terraform Workflow



4. Install Terraform

Linux (Ubuntu)

```
sudo apt update
sudo apt install \
  terraform
terraform version
```

Windows

```
choco install
terraform
terraform
version
```

macOS

```
brew install
terraform
terraform
version
```

Verify Installation

```
$ terraform version
Terraform v1.6.0
on linux_amd64
```

5. Basic Terraform Commands

```
# Initialize the working directory
terraform init

# Format configuration files
terraform fmt

# Validate the configuration
terraform validate

# Show planned changes
terraform plan

# Apply the changes
terraform apply

# Destroy the resources
terraform destroy
```

6. Example: Local Configuration

main.tf (create a local file)

```
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.4"
    }
  }
}

resource "local_file" "example" {
  filename = "homelab-note.txt"
  content  = "Hello from Terraform!\n"
}
```

7. Run and Inspect the Changes

1. Create a new directory and add the main.tf file.
2. Run `terraform init` to download the provider.
3. Run `terraform validate` to check syntax.
4. Run `terraform plan` to see what will be created.
5. Run `terraform apply` to create the file.
6. Confirm the file exists.
7. Run `terraform destrory` to remove it.

```
$ terraform init
Initializing...

$ terraform plan
+ resource "local_file" "example" will be created

$ terraform apply
Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

$ cat homelab-note.txt
Hello from Terraform!

$ terraform destroy
Destroy complete! Resources: 1 destroyed.
```

8. Plan vs Apply

terraform plan

- Shows what will change.
- Does not create or modify resources.
- Helps you review changes before applying.
- Reduces the risk of unintended actions.

terraform apply

- Creates or modifies resources.
- Executes the planned changes.
- Should be reviewed carefully before running.
- Can create real cloud resources, which may incur cost.

9. Important Notes

- Terraform state can contain sensitive information.
- Do not commit terraform.tfstate to public repositories.
- Use .gitignore to exclude state files.

- Cloud resources require proper permissions.
- Always review the plan output.
- Be aware of potential costs.
- Use least privilege access.

10. Practical Exercise

1. Create a new directory (e.g., terraform-local).
2. Add the example main.tf file.
3. Run `terraform init`, `terraform plan`, and `terraform apply`.
4. Check that `homelab-note.txt` is created.
5. Inspect the file content.
6. Run `terraform destroy` to clean up.

11. Security Reminder

- Never commit sensitive Terraform state.
- Do not commit cloud credentials or API keys.
- Use environment variables or a secure secret manager.
- Review permissions before applying changes.
- Be careful when using real cloud providers to avoid unexpected costs.

Junior Engineer Tip

- Always run terraform plan before terraform apply.
- Check the output carefully.
- Use version control for your .tf files.
- Keep your state file private and secure.

Common Mistakes

- Committing terraform.tfstate to Git.
- Applying changes without reviewing the plan.
- Using overly broad cloud permissions.
- Accidentally creating costly cloud resources.
- Not cleaning up test resources.

Production Connection

- Infrastructure as Code makes infrastructure repeatable and more reliable.
- The same skills are used in real cloud environments (e.g., AWS, Azure, GCP).
- Proper planning, state management, and access control are essential in production.

Backups, Recovery, Security, and Lab Maintenance

1. Why Backups and Recovery Matter?

- Protects your data from accidental loss, corruption, or system failure.
- Allows you to restore and continue working quickly.
- Builds confidence that your environment is recoverable.
- A key skill for real-world DevOps and production systems.

2. Backup vs Restore

Term	What It Means
Backup	A copy of data, files, or configuration that can be used later.
Restore	The process of using a backup to recover data.
Backup Success	A backup is only useful if you can successfully restore it.
Test Recovery	Always test that your backup can be restored.

3. Back Up PostgreSQL Database

```
# Create a backup
docker exec -t homelab-db \
  pg_dump -U appuser -d homelab > \
  backups/homelab_backup.sql

# Check the backup file
ls -lh backups/
```

This creates a full database backup inside the backups/ directory.

4. Restore the Backup (Test Recovery)

```
# Create a new test database
docker exec -it homelab-db \
  psql -U appuser -c "CREATE DATABASE
  homelab_test;"

# Restore the backup
docker exec -i homelab-db \
  psql -U appuser -d homelab_test < \
  backups/homelab_backup.sql

# Verify the data
docker exec -it homelab-db \
  psql -U appuser -d homelab_test -c \
  "SELECT COUNT(*) FROM your_table;"
```

5. Manage Secrets and Environment Files

- Keep sensitive information out of your code repository.
- Use environment variables or a .env file (for local use only).
- Do not commit .env files to Git.
- Use strong passwords and unique values.
- Consider using a secrets manager in real production environments.

Example .env file (do not commit)

```
POSTGRES_USER=appuser
POSTGRES_PASSWORD=strongpassword
POSTGRES_DB=homelab
APP_SECRET=change_this_value
```

6. Update Dependencies and Images

- Keep your application dependencies updated.
- Pull the latest container images.
- Rebuild and redeploy regularly.
- Check for security updates.

```
# Pull latest images
docker compose pull

# Rebuild and start
docker compose up -d --build
```

7. Review Exposed Ports

- Only expose the ports you need.
- Avoid exposing databases to your host unless required.
- Use Docker networks for internal communication.

```
# Check exposed ports
docker ps

# Example output
PORTS
0.0.0.0:8080->8080/tcp   # needed
0.0.0.0:5432->5432/tcp   # consider removing
0.0.0.0:3000->3000/tcp   # needed
```

8. Resource Limits and Disk Cleanup

- Monitor CPU, memory, and disk usage.
- Set resource limits to prevent a single container from using all resources.
- Remove unused containers, images, and volumes.

```
# Check resource usage
docker stats

# Clean up unused resources
docker system prune -a

# Check disk usage
docker system df
```

9. Stop Unused Services

- Stop services you are not using.
- Remove unneeded containers and volumes.
- Delete any cloud resources you created for testing to avoid unexpected costs.

```
# Stop all services
docker compose down

# Remove unused volumes (if not needed)
docker volume prune
```

10. Simple Lab Operations Checklist

- [x] Back up important data regularly.
- [x] Test that you can restore the backup.
- [x] Keep secrets out of Git.
- [x] Update dependencies and container images.
- [x] Review exposed ports.
- [x] Monitor resource usage and clean up.
- [x] Stop unused services.
- [x] Keep your lab documentation up to date.

11. Practical Exercise

1. Create a PostgreSQL backup using the command in section 3.
2. Restore the backup to a new test database using section 4.
3. Verify that the expected records exist.
4. Update one container image to the latest version.
5. Check and remove unused Docker resources.
6. Complete the lab operations checklist.

Security Reminder

- Never commit sensitive data, including Terraform state, passwords, or API keys.
- Use environment variables or a secure secrets manager.
- Restrict file permissions (e.g., chmod 600 .env).
- Review permissions before sharing your project.

Common Failure

- Backup file is created but not tested.
- Restore fails due to wrong credentials or database name.
- Secrets are committed to Git.
- Disk space runs out and containers stop.
- Unnecessary ports are exposed.

Junior Engineer Tip

- A backup is only useful when you can successfully restore it.
- Check your backups regularly.
- Keep your lab simple and affordable.
- Remove resources you no longer need.
- Good maintenance prevents many problems.

Production Awareness

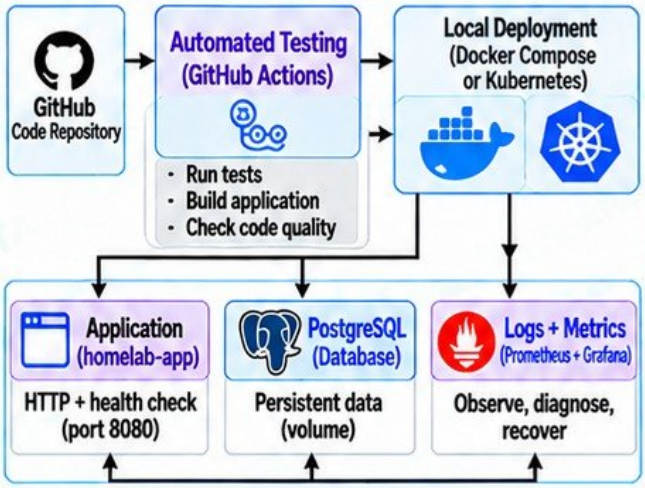
- Backups, security, and maintenance reduce downtime and data loss.
- Regular updates and cleanup improve stability and security.
- The same principles apply in production environments at a larger scale.

Final Project: Operate Your Home Lab Like a Junior DevOps Engineer

1. Project Overview

- Bring everything together in one complete project.
- Demonstrate real DevOps skills using your local environment.
- Build, test, deploy, monitor, troubleshoot, recover, and document.
- Use the same homelab-app throughout the project.

2. Final Architecture (Local Environment)



3. Tools Used in the Project

- GitHub (source code)
- GitHub Actions (CI)
- Docker / Docker Compose
- Kubernetes (optional)
- PostgreSQL (database)
- Prometheus (metrics)
- Grafana (dashboard)
- Docker logs / metrics
- Documentation (README)

4. Final Engineering Challenge

1. Push an application update to GitHub.
2. Confirm automated tests pass.
3. Deploy the updated application locally.
4. Verify the application and database.
5. Inspect logs and resource usage.
6. Simulate a controlled application failure.
7. Diagnose and recover the service.
8. Restore data from a tested backup.
9. Document the architecture and troubleshooting process.

5. Example Project Structure

```
homelab-app/
├── .github/
│   └── workflows/
│       └── ci.yml
├── app/                    # application code
├── docker/                 # Dockerfile
├── docker-compose.yml      # app, db, monitoring
├── k8s/                    # (optional) manifests
├── scripts/                # deployment, backup
├── README.md               # documentation
└── .env.example            # environment variables
```

6. Key Commands (Examples)

Run automated tests (GitHub Actions)

```
git add .
git commit -m "Update application"
git push
```

Deploy locally (Docker Compose)

```
docker compose up -d --build
```

Check application and database

```
curl http://localhost:8080/health
docker compose exec db psql -U appuser -d homelab-db
SELECT COUNT(*) FROM your_table;
```

7. Verify the Running Application

http://localhost:8080/health

```
{
  "status": "healthy",
  "version": "1.0.0"
}
```

- Application is accessible.
- Health check returns 200 OK.
- Version matches your latest deployment.

8. Monitor and Inspect

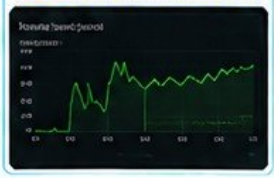
View logs

```
docker compose logs -f app
```

Check resource usage

```
docker stats
```

Prometheus (http://localhost:9090)



Grafana (http://localhost:3000)



9. Simulate a Failure and Recover

1. Intentionally break the application (e.g., wrong port or bad configuration).
2. Observe the failure in logs and health check.
3. Identify the root cause.
4. Apply a fix and redeploy.
5. Verify the application is healthy again.

10. Backup and Restore Data

```
# Create a backup
docker compose exec db \
  pg_dump -U appuser -d homelab-db > backup.sql
# Restore to a new test database
docker compose exec -T db \
  psql -U appuser -d homelab_test < backup.sql
```

11. Publish a Professional README

- Project overview and architecture diagram.
- Setup instructions (prerequisites).
- How to run tests, deploy, and verify.
- How to back up and restore data.
- Troubleshooting guide with real examples.
- Cleanup steps.
- Known limitations.
- Your learning summary.

12. Completion Standard

- Can explain what each component does.
- Can demonstrate the working application.
- Can investigate a failure using logs and metrics.
- Can recover the service.
- Can restore data from a backup.
- Can repeat the setup using your own documentation.

Production Awareness

- Follow the same engineering practices used in real environments.
- A backup is only useful when you can successfully restore it.
- Monitoring helps you detect problems before users report them.
- Good documentation makes your work easier to maintain and share.